Prediction of Added Mass Force Using Neural Networks

April 9, 2026

1 Introduction

Added Mass Concept When a body accelerates in a fluid, it must also accelerate the surrounding fluid. This results in an additional force:

$F = (m + m_a)a$

where: 1) m = solid mass, 2) m_a = added mass, 3) a = acceleration

Added mass depends on:

1) Geometry (e.g., cylinder, sphere, concave/convex nose)
2) Fluid properties
3) Velocity and acceleration
4) Free-surface effects (important in water entry problems)

Role of Machine Learning Instead of solving governing equations repeatedly, we aim to:

1) Learn mapping:
2) Replace expensive simulations with:
   (a) Data-driven surrogate models
   (b) Neural networks

[1]:
```
!module load cuda/12.6
!module load cudnn/8.9.7.29-12-cuda12.6
import torch
print("CUDA version:", torch.version.cuda)
print("cuDNN version:", torch.backends.cudnn.version())
print("GPU available?", torch.cuda.is_available())
print("Device count:", torch.cuda.device_count())
print("Device name:", torch.cuda.get_device_name(0))
import torch.nn as nn
import pandas as pd
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import random_split
import numpy as np
```

```
Loading cuda version 12.6
Loading cuda version 12.6
Loading cudnn version 8.9.7.29-12
```

```
CUDA version: 12.6
cuDNN version: 91002
GPU available? True
Device count: 1
Device name: NVIDIA A16
```

2 Dataset Description

Dataset contains 50,000 samples

1) Input features: x_1, x_1, x_3 (represent velocity, fluid property, and geometry)
2) Output: y (added mass force)

Data split:

```
80% training
10% validation
10% testing
```

[2]:
```
data = torch.tensor(
    pd.read_csv("complex_regression_data.csv").values,
    dtype=torch.float32
)

X = data[:, :-1]
y = data[:, -1].unsqueeze(1)

dataset = TensorDataset(X, y)
```

[3]:
```
N = len(dataset)
n_train = int(0.8 * N)
n_val   = int(0.1 * N)
n_test  = N - n_train - n_val

train_ds, val_ds, test_ds = random_split(
    dataset,
    [n_train, n_val, n_test],
    generator=torch.Generator().manual_seed(42)  # reproducible
)
```

3 Neural Network Architecture

A Multi-Layer Perceptron (MLP) is used:

1) Input layer: 3 neurons
2) Hidden layers: 128 neurons (ReLU), 128 neurons (ReLU), 64 neurons (ReLU)
3) Output layer: 1 neuron

Activation: ReLU (nonlinear modeling capability)

[4]:
```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(3, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        return self.net(x)
```

4 Training Strategy

1) Optimizer: Adam (Optax)
2) Learning rate: $10^{-}5$
3) Loss function: Mean Squared Error (MSE)
4) Mini-batch training (batch size = 128)
5) Early stopping used to prevent overfitting

[5]:
```
model = MLP()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-5)
```

[7]:
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

#  Compile the model (PyTorch 2.0+)
model = torch.compile(model, backend="inductor")  # or backend="nvfuser" on GPU

#  DataLoaders with multiple workers
train_loader = DataLoader(train_ds, batch_size=64, shuffle=True, num_workers=4,␣
 ↪pin_memory=True)
val_loader   = DataLoader(val_ds, batch_size=64, shuffle=False, num_workers=4,␣
 ↪pin_memory=True)
test_loader  = DataLoader(test_ds, batch_size=64, shuffle=False)
```

[6]:
```
epochs = 5000
patience = 500
best_val = float('inf')
wait = 0
```

```python
best_model_state = None

for epoch in range(epochs):
    #  Training loop
    model.train()
    for xb, yb in train_loader:
        xb, yb = xb.to(device, non_blocking=True), yb.to(device,␣
↪non_blocking=True)
        optimizer.zero_grad()
        pred = model(xb)
        loss = criterion(pred, yb)
        loss.backward()
        optimizer.step()

    #  Validation every 100 epochs
    if epoch % 100 == 0:
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for xb, yb in val_loader:
                xb, yb = xb.to(device, non_blocking=True), yb.to(device,␣
↪non_blocking=True)
                val_loss += criterion(model(xb), yb).item()
            val_loss /= len(val_loader)


        print(f"Epoch {epoch}: Test Loss = {loss:.4e} Val Loss = {val_loss:.4e}")

        #  Early stopping
        if val_loss < best_val:
            best_val = val_loss
            best_model_state = model.state_dict()
            torch.save(model.state_dict(), "best_model.pth")
            wait = 0
        else:
            wait += 100

        if wait >= patience:
            print("Early stopping triggered")
            break

#  Restore best model
if best_model_state is not None:
    model.load_state_dict(best_model_state)
```

```
Epoch 0: Test Loss = 1.7616e+02 Val Loss = 1.5267e+02
Epoch 100: Test Loss = 4.4485e-01 Val Loss = 3.7480e-01
```

```
Epoch 200: Test Loss = 2.7846e-01 Val Loss = 3.4919e-01
Epoch 300: Test Loss = 2.8138e-01 Val Loss = 3.4433e-01
Epoch 400: Test Loss = 2.3610e-01 Val Loss = 3.4033e-01
Epoch 500: Test Loss = 3.4326e-01 Val Loss = 3.3731e-01
Epoch 600: Test Loss = 2.7813e-01 Val Loss = 3.3981e-01
Epoch 700: Test Loss = 3.0386e-01 Val Loss = 3.3714e-01
Epoch 800: Test Loss = 3.4424e-01 Val Loss = 3.3526e-01
```

[9]:
```
model.load_state_dict(torch.load("best_model.pth"))
```

[9]:
```
<All keys matched successfully>
```

5 Results

Training converges steadily

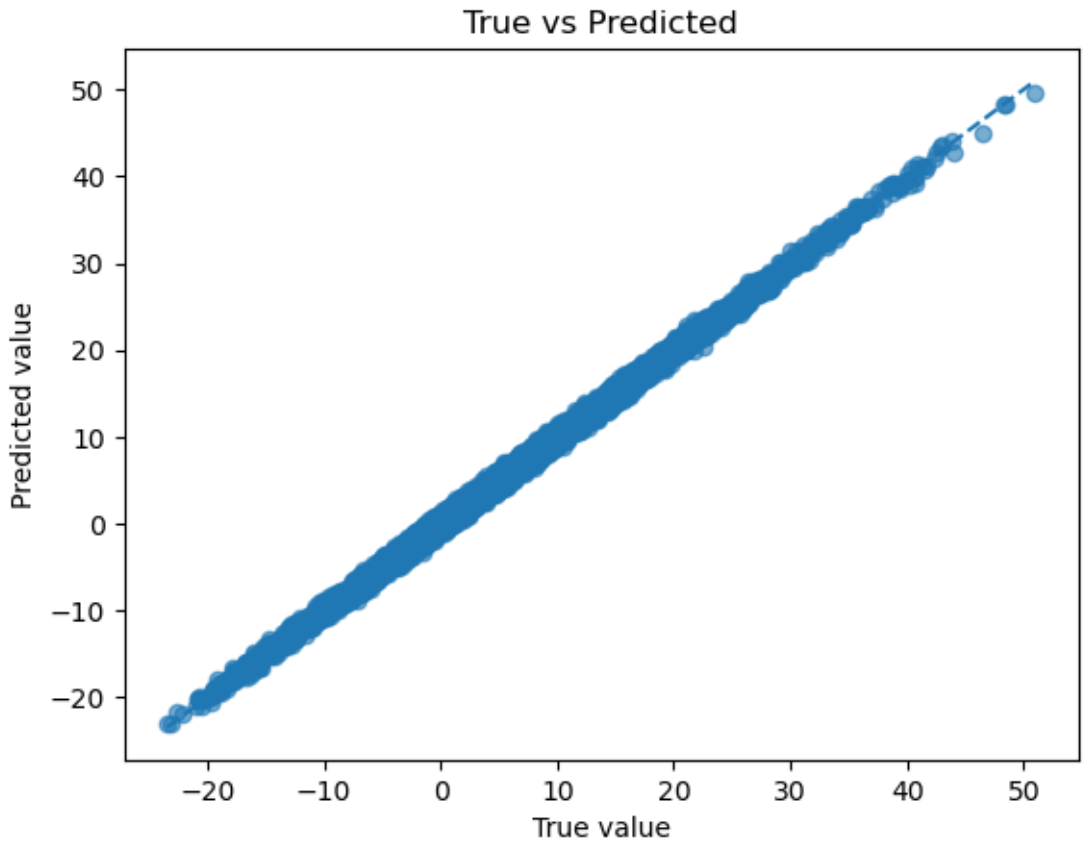
1) Model accuracy: $R^2 = 0.9975$
2) The scatter plot shows: Predicted vs true values align almost perfectly which indicates excellent generalization

[10]:
```
model.eval()

y_true = []
y_pred = []

with torch.no_grad():
    for xb, yb in test_loader:
        xb = xb.to(device)
        preds = model(xb)

        y_true.append(yb.cpu().numpy())
        y_pred.append(preds.cpu().numpy())

y_true = np.vstack(y_true)
y_true = np.squeeze(y_true)
y_pred = np.vstack(y_pred)

test_loss = np.mean((y_true - y_pred)**2)
print(f"Test Loss = {test_loss:.4e}")
```

```
Test Loss = 3.3854e-01
```

[12]:
```
import matplotlib.pyplot as plt
plt.figure()
plt.scatter(y_true, y_pred, alpha=0.6)
plt.plot([y_true.min(), y_true.max()],
         [y_true.min(), y_true.max()], '--')
plt.xlabel("True value")
```

```
plt.ylabel("Predicted value")
plt.title("True vs Predicted")
plt.show()
```



[13]:
```
SS_res = np.sum((y_true - y_pred)**2)
SS_tot = np.sum((y_true - np.mean(y_true))**2)

R2 = 1 - SS_res / SS_tot
print("R² =", R2)
```

```
R² = 0.99729973
```

[ ]:
```

```